

第4章 串、数组和广义表



4.1 串类型的定义

4.2 串的实现和表示

4.3 串的模式匹配算法——KMP算法

4.4 数组

4.5 广义表

4.6 案例分析与实现

4.1 串的定义(P105)

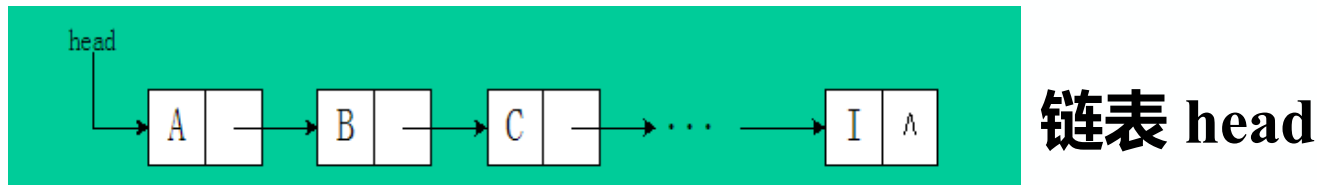
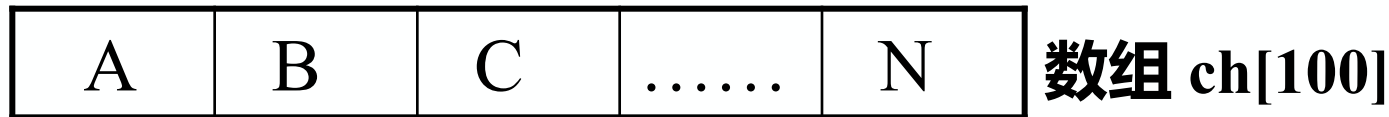
串（或字符串）(String)----是由零个或多个字符组成的有限序列。是特殊的、**内容受限的线性表**，特殊性体现在数据元素是一个字符。
字母、数字、其他字符

$S = "a_1 a_2 \cdots a_n"$

n=0 空串

S: **串名** $a_1 a_2 \cdots a_n$: **串值** n: **串长** ≥ 0

串的存储结构----顺序存储和链式存储



4.1 串的定义(P105)

子串：一个串中任意个连续字符组成的子序列（含空串）称为该串的子串。

例如，"abcde" 的子串有：

""、'a'、"ab"、"abc"、"abcd"和"abcde"等

主串：包含子串的串相应地被称为主串。

真子串是指不包含自身的所有子串。

4.1 串的定义(P105)

串相等：两个串的长度相等，并且各个对应位置上的字符都相同。

如：

`"abcd"` $\neq$ `"abc"`

`"abcd"` $\neq$ `"abcde"`

所有空串是相等的。

4.1 术语大集合

串长：串中字符的数目 n 称为串的长度，引号不算在其中

字符位置：字符在序列中的序号为该字符在串中的位置

子串位置：子串第一个字符在主串中的位置

空串(null string)：包含零个字符的串，它的长度为零。为了清楚起见，一般用符号“ $\emptyset$ ”来表示“空串”

空格串：由一个或多个空格组成的串，它的长度为串中空格的个数，与空串不同

例如：假设 a 、 b 、 c 、 d 为如下的4个串：

$a = \text{"BEI"}$ $b = \text{"JING"}$ $c = \text{"BEIJING"}$ $d = \text{"BEI JING"}$

则它们的长度分别是3、4、7和8

并且 a 和 b 都是 c 和 d 的子串，

a 在 c 和 d 中的位置都是1，而 b 在 c 中的位置是4，在 d 中的位置则是5

串是一种特殊的线性表，其特殊性体现在（ ）。

- A. 可以顺序存储**
- B. 数据元素是字符**
- C. 可以链式存储**
- D. 数据元素可以是数字**

答案： B

下面关于串的叙述中，（ ）是不正确的？

- A. 串是字符的有限序列
- B. 空串是由空格构成的串
- C. 模式匹配是串的一种重要运算
- D. 串既可以采用顺序存储，也可以采用链式存储

答案：B

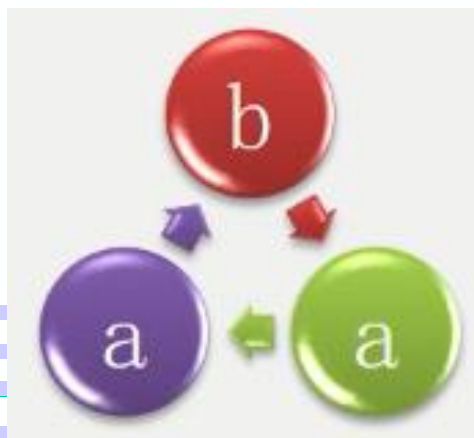
解析：空格常常是串的字符集合中的一个元素，有一个或多个空格组成的串称为空格串，零个字符的串称为空串，空串的长度为零。

病毒感染检测—案例引入

研究者将人的DNA和病毒DNA均表示成由一些字母组成的字符串序列。通过分析发现，**病毒的DNA序列是环状的，而人的DNA序列是线性的。**

然后检测某种病毒DNA序列是否在患者的DNA序列中出现过，如果出现过，则此人感染了该病毒，否则没有感染。

例如，假设病毒的DNA序列为baa，患者1的DNA序列为aaabbba，则感染，患者2的DNA序列为babbbba，则未感染。



aba baa aab 都是病毒DNA序列

病毒感染检测—案例引入

10个待检测数据，左侧为病毒序列，右侧为患者DNA序列

病毒感染检测输入数据.txt - 记事本

文件(F) 编辑(E) 格式(O) 查看(V)

```
10
baa      bbaabbba
baa      aaabbbba
aabb     abceaabb
aabb     abaabcea
abcd     cdabbbab
abcd     cabbbbab 无
abcde    bcdedbda 无
acc      bdedbcda 无
cde      cdcdec
cced     cdccdcce
```

病毒感染检测输出结果.txt - 记事本

文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

```
baa      bbaabbba      YES
baa      aaabbbba      YES
aabb     abceaabb      YES
aabb     abaabcea      YES
abcd     cdabbbab      YES
abcd     cabbbbab      NO
abcde    bcdedbda      NO
acc      bdedbcda      NO
cde      cdcdec       YES
cced     cdccdcce      YES
```

4.3 串的抽象类型定义 (P107)

ADT String{

数据对象: $D = \{a_i \mid a_i \in \text{CharacterSet}, i=1, 2, \dots, n, n \geq 0\}$

数据关系: $R1 = \{ \langle a_{i-1}, a_i \rangle \mid a_{i-1}, a_i \in D, i=2, \dots, n \}$

基本操作:

StrAssign(&T, chars) // 串赋值, 生成值为chars的串T

StrCompare(S, T) // 串比较, $S > T: 1$, $S = T: 0$, $S < T: -1 \dots$

StrLength(S) // 求串长, 即返回串S中的元素个数

Concat(&T, S1, S2) // 串连接, 用T返回S1+S2的新串

**SubString(&Sub, S, pos, len) // 求S中第pos个位置开始的
长度为len的子串, 通过Sub串返回**

.....

Index(S, T, pos) // 子串定位函数(模式匹配), 返回位置

Replace(&S, T, V) // 用串V替换主串S中的子串T

} ADT String

4.3 串的函数应用举例

例1：设 s ="I AM A STUDENT.", t ="GOOD", q ="WORKER"
求：

StrLength(s) = 15

StrLength(t) = 4

SubString($\&sub$, s , 8, 7) = "STUDENT"

SubString($\&sub$, t , 2, 2) = "OO"

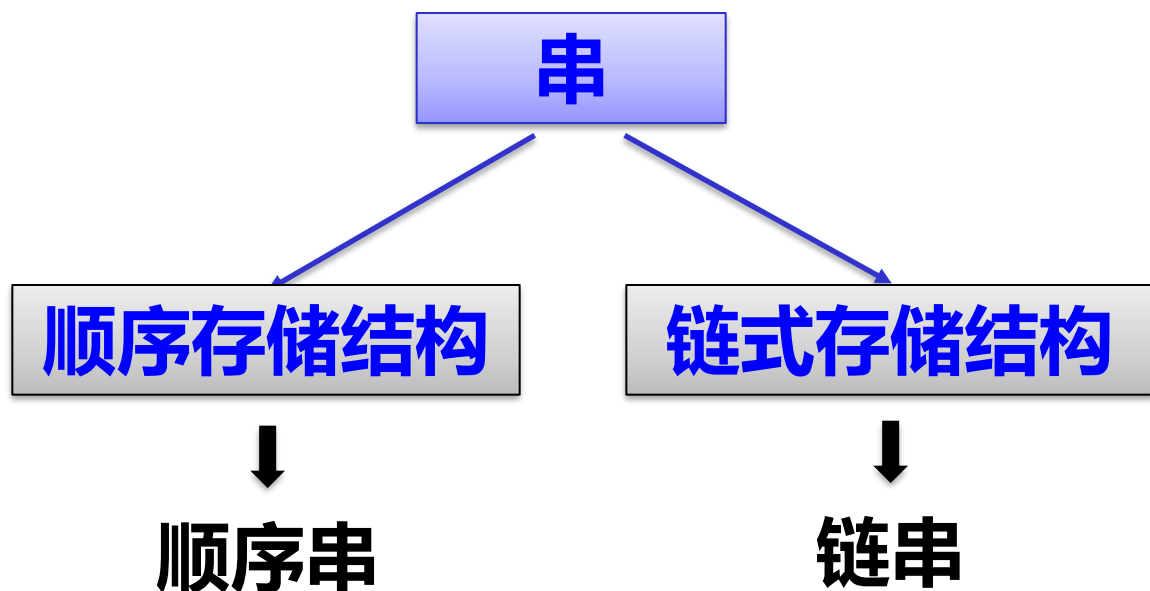
Index(s , "AM") = 3

Index(s , t) = 0 // s 中没有"GOOD"

Replace($\&s$, "STUDENT", q) = "I AM A WORKER."

4.3.2 串的存储结构 (P108)

串中元素逻辑关系与线性表的相同，串可以采用与线性表相同的存储结构。



逻辑结构



存储结构

串的顺序存储 (P108)

将串中字符顺序地存入连续的存储单元中;

```
#define    MAXLEN    225 //串的最大长度固定为225, 最多存放225个字符  
typedef struct  
{  
    char ch[MAXLEN+1]; //字符类型, ch是存储串值的一维数组  
    int length; //串的实际长度  
} SString; //顺序串
```

SString S; **S.ch[1]** **S.length 等同于S.ch[0]**

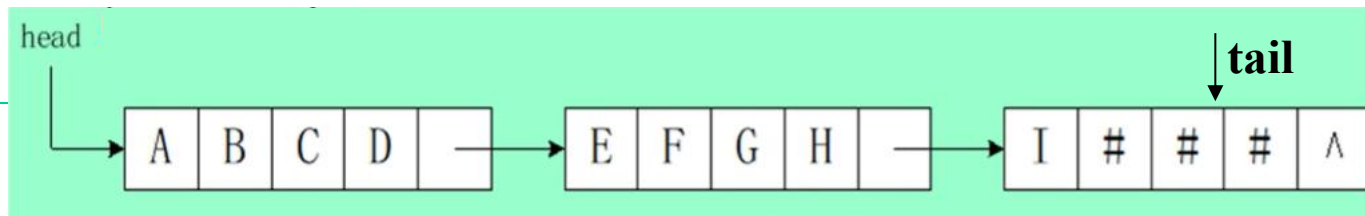
为了便于说明问题, 本章中 串都是从下标为1开始存储, 串值存储在1 ~ length的下标上, 注意区分之前 0 ~ length-1 的数组下标

下标为0存储串长

串的链式存储 (P109)

串的链式存储——块链结构，不带头结点的单链表存储串。
为了便于串的操作，附设头尾指针分别指向第一个和最后一个结点
其类型定义如下：

```
#define CHUNKSIZE 80 //宏定义块大小，单个块最多存储80个字符
typedef struct Chunk
{
    char ch[CHUNKSIZE]; //存放字符块
    struct Chunk *next; //指针域
} Chunk; //块结点
typedef struct
{
    Chunk *head,*tail; //串的头和尾指针
    int length; //串的实际长度
} LString; //链串，串的块链结构
```



4.3.3 串的模式匹配算法 (P109)

模式匹配： 确定子串第一个字符在主串中出现的位置（定位）称为模式匹配。模式匹配成功是指在主串S中能够找到子串T，返回位置；否则称子串T在主串S中不存在，返回0。

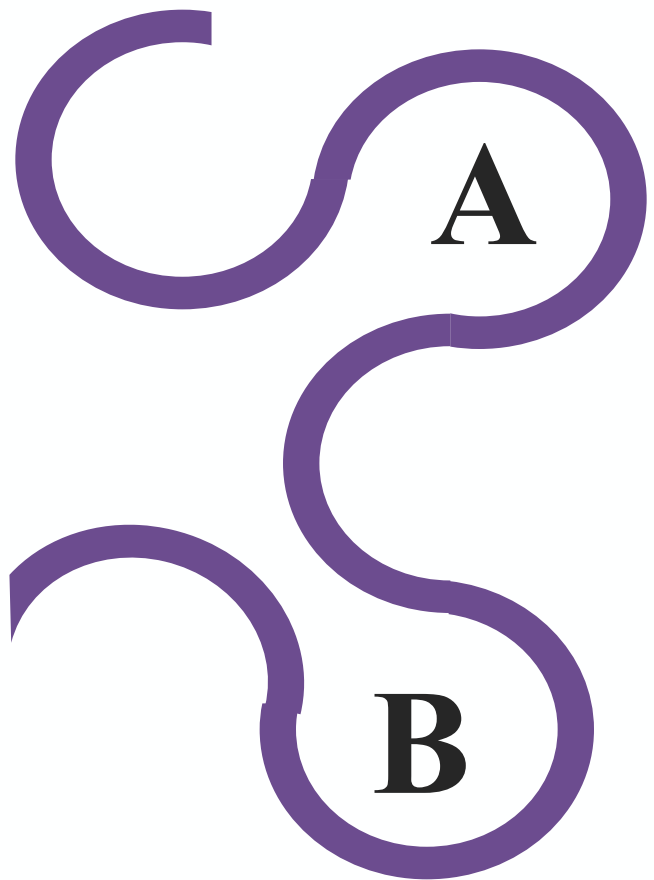
例如：设主串（正文串）为ababc**abcac**bab

设子串（模式串）为**abcac**

则称模式匹配成功，返回的定位为6。

即子串的第1个字符在主串中出现的位置。

4.3.3 串的模式匹配算法 (P110)



算法应用:

搜索引擎、拼写检查、语言翻译

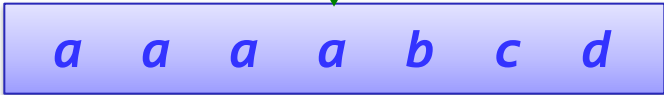
算法种类:

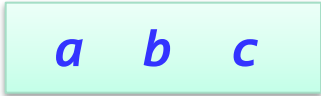
- BF算法 (又称古典的、经典的、朴素的、穷举的)
- KMP算法 (特点: 速度快)

4.3.3 BF算法 (P110)

Brute-Force 简称为**BF算法**，**BF是指暴力的意思！** 亦称简单匹配算法。采用穷举的思路。

例如，S="aaaabcd"，T="abc"。

主串(正文串) S: 

子串(模式串) T:  **匹配成功**

BF算法思路：从主串S指定的字符开始（一般是第一个）和子串T中的第一个字符比较。

若相等，则继续逐个比较后续字符，直到子串T中的每个字符依次和主串S中一个连续字符序列相等，扫描完子串，则称匹配成功；若比较过程中有某对字符不相等，则从主串S的下一个字符起，再重新和子串T的第一个字符比较。如果S中的字符都比完了，仍然没有匹配成功，则称匹配失败。

4.3.3 BF算法 (P89)

匹配第四趟, $i=7$



主串(正文串) S: a a a a b c d

子串(模式串) T: a b c

匹配成功

$j=4$

- 变量 i 和 j 分别扫描字符串S和T, 初始时刻 $i=j=1$
- 对于每个 $S[i]$, $j=1$ 开始比较。若匹配不相等,
 $i=i-j+2$ (回溯); $j=1$ (从头开始)

◆ 循环条件是串S或者串T全部走一遍

$7-3=4$ 子串第一个字符
在主串中出现的序号

- 若T扫描完毕, 则返回序号 $i-T.length$

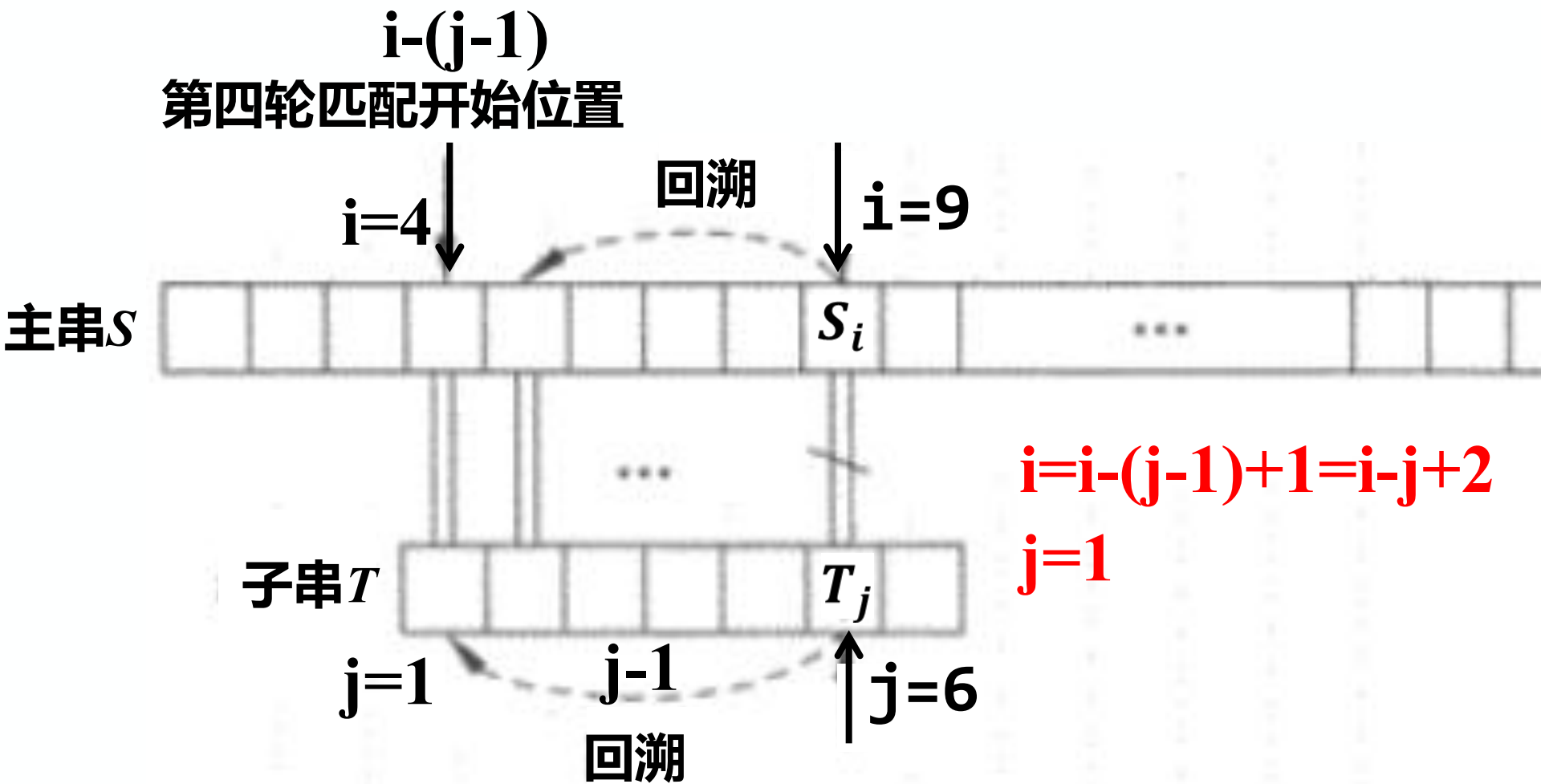
- 若S全部比较完都没有完全匹配上的, 则返回0

成功

失败

为了便于说明问题, 本章中字符串存储在下标为 $1 \sim length$ 的位置上

4.3.3 BF回溯 (P89)



Index(S, T, pos)

将主串的第pos个字符和模式串的第1个字符比较，

- 若**相等**，继续逐个比较后续字符；
- 若**不等**，从主串的下一字符起，重新与模式串的第一个字符比较。

直到主串的一个连续子串字符序列与子串相等。返回值为S中与T匹配的子序列**第一个字符的序号**，即匹配成功。

否则，匹配失败，返回值 0

4.3.3 BF算法(P111, 算法4.1)

```
int Index_BF(SString S, SString T, int pos)
{ //从主串S的第pos个字符开始, 与子串T的第一个字符比较
    int i=pos, j=1; //从主串中间的某个pos位置开始匹配
    while(i<=S.length && j<=T.length) //大于串长跳出循环
    {
        if (S.ch[i] == T.ch[j])
            { ++i; ++j; } //主串子串继续比较后续字符
        else
            { i=i-j+2; j=1; } //主串子串的指针回溯, 重新开始匹配
    }
    if (j>T.length) return i-T.length; //成功返回匹配的第一个字符下标
    else return 0; //模式匹配不成功
}
```

主串S: abaaaaab 子串T: aba

例: S="00000000000000000000000001", T="0001", pos=1

每趟不成功的匹配都发生在子串的最后一个字符与主串中相应字符的比较。此时 时间复杂度出现最坏情况。

若n为主串长度, m为子串长度, 最坏情况是
主串前面n-m个位置都部分匹配到子串的最后一位, 即这n-m位各比较了m次, 最后一个m还要再比较一次。

$$(n-m) \times m + m \times 1$$

- 最坏情况下的时间复杂度为 $O(n \times m)$ 。(n次遍历T)
当m远小于n
- 最好情况下的时间复杂度为 $O(m)$ 。(1次遍历T)
- 平均的时间复杂度为 $O(n \times m)$ 。

4.3.3 串的模式匹配算法——KMP算法(P112)

KMP算法是D.E.Knuth、J.H.Morris和V.R.Pratt共同提出的，简称**KMP算法**。

该算法较BF算法有较大改进，主要是**消除了主串指针的回溯**，从而使算法效率有了某种程度的提高。

改进：BF算法每趟匹配过程中出现字符比较不等时，

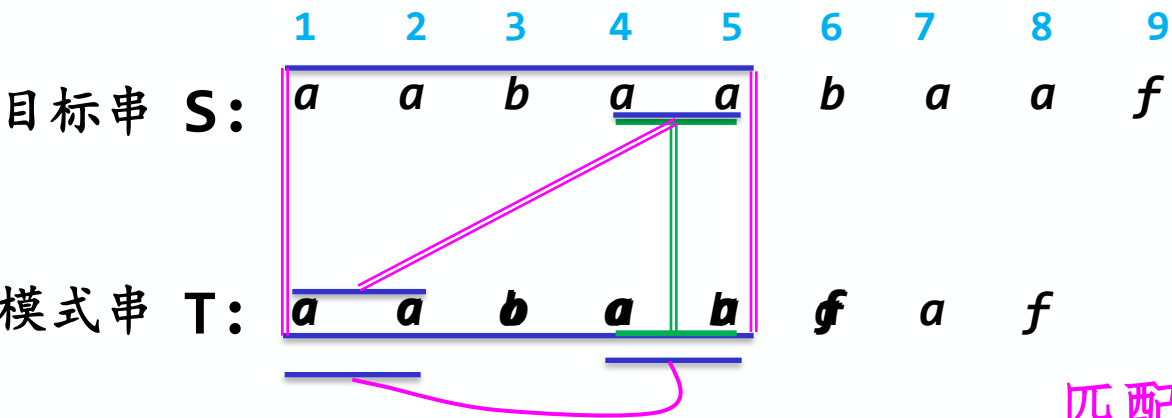
```
if (S[i] == T[j])  
    { ++i; ++j; }  
else  
    { i=i-j+2; j=1; }
```

不回溯指针i，并利用已得到的“部分匹配”结果将子串向右“滑动”尽可能远的一段距离后，继续进行比较。

4.3.2 BF与KMP

目标串S=“aabaabaaf”，模式串T=“aabaaf”

KMP



匹配成功

前缀表

aabaa { 前缀: a aa aab aaba
 后缀: a aa baa abaa

最长相等的前后缀长度=2，下一轮匹配从子串T的3号位开始

4.3.2 前缀、后缀

目标串 $S = \text{"aabaabaaf"}$ ，模式串 $T = \text{"aabaaf"}$

前缀：包含首字符，不包含尾字符的所有子串，即不包含它本身。

后缀：包含尾字符，不包含首字符的所有子串，即不包含它本身。

abcd { 前缀: a ab abc
后缀: d cd bcd

aabaaf { 前缀: a aa aab aaba aabaa
后缀: f af aaf baaf abaaf

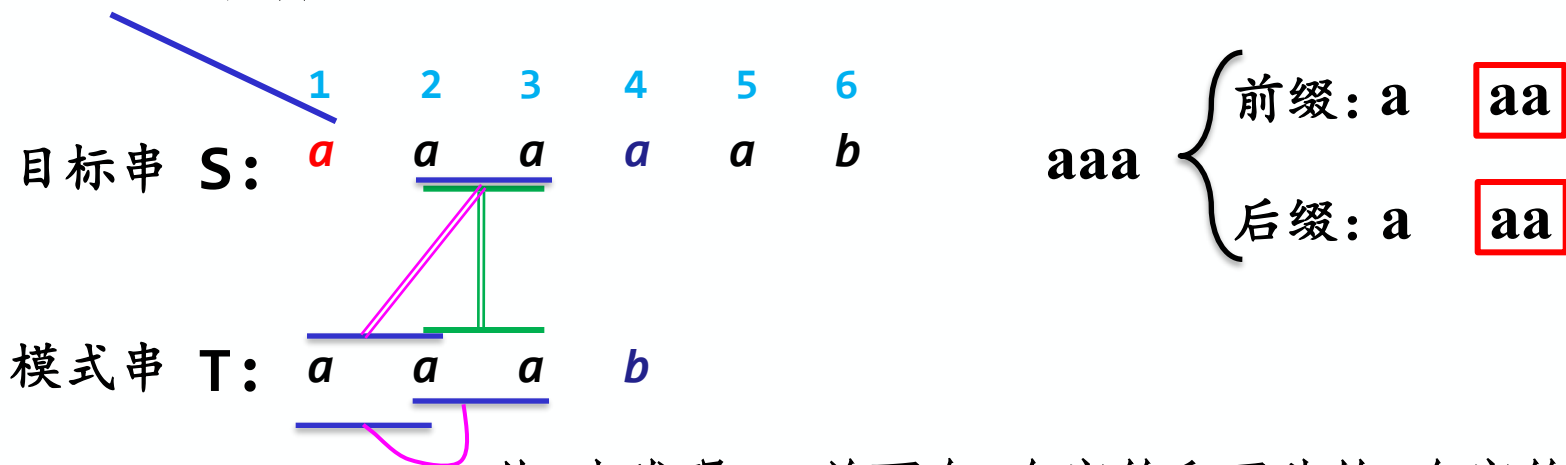
最长相等的前后缀长度 (可以交叉)

over、end就是lover、friend这些单词字符串的后缀。

4.3.2 KMP算法用next数组保存部分匹配信息

目标串 $S = \text{"aaaaab"}$ ，模式串 $T = \text{"aaab"}$

开始匹配的字符



从 T 中发现: b 前面有 2 个字符和开头的 2 个字符相同



T 中每个位置都有这种信息, next[4] = 3

在某一趟, 当 S 的第 i 个字符与 T 的第 j 个字符失配时, T 中下一趟开始比较的位置, 就存放在 $\text{next}[j]$ 中 (当遇到冲突、失配后退到的位置)

next[j]函数表明当子串中第j个字符与主串中相应字符“失配”时，在子串中需重新和主串中该字符进行比较的字符位置。

对于模式串 $t = "t_1 \cdots t_j \cdots t_{m-1}"$

前缀

从头开始的
 $k-1$ 个元素

后缀

t_j 前面的
 $k-1$ 个元素

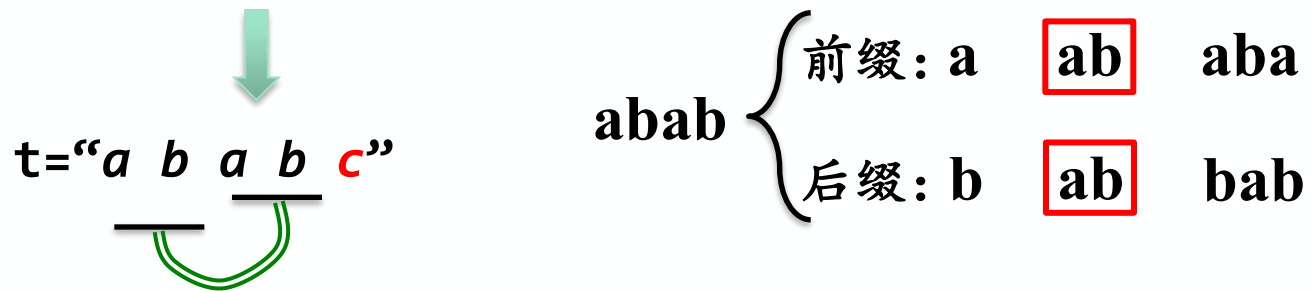
$$\text{next}[j] = \begin{cases} \text{Max}\{ k \mid 1 < k < j, \text{ 且 } "t_1 t_2 \cdots t_{k-1}" = "t_{j-k+1} \cdots t_{j-1}" \} & \text{当此集合非空时} \\ 0 & \text{当 } j=1 \text{ 时} \\ 1 & \text{其他情况} \end{cases}$$

1 next[j]是指t_j字符前有多少个字符与t开头的字符相同

模式串t中字符t_j, 存在某个k (1<k<j), 使得以下成立:

$$\underbrace{"t_1 \cdots t_{k-1}"}_{\text{开头的}k-1\text{个字符}} = \underbrace{"t_{j-k+1} \cdots t_{j-1}"}_{t_j\text{前面的}k-1\text{个字符}}$$

例如, t="a b a b c" 考虑t[5]='c' 失配



⇒ k=3, 所以next[5] = k = 3。

2

$\text{next}[j]$ 是指 t_j 字符前最多有多少个字符与 t 开头的字符相同

模式串 t 中字符 t_j , 存在某个 k ($1 < k < j$), 使得以下成立:

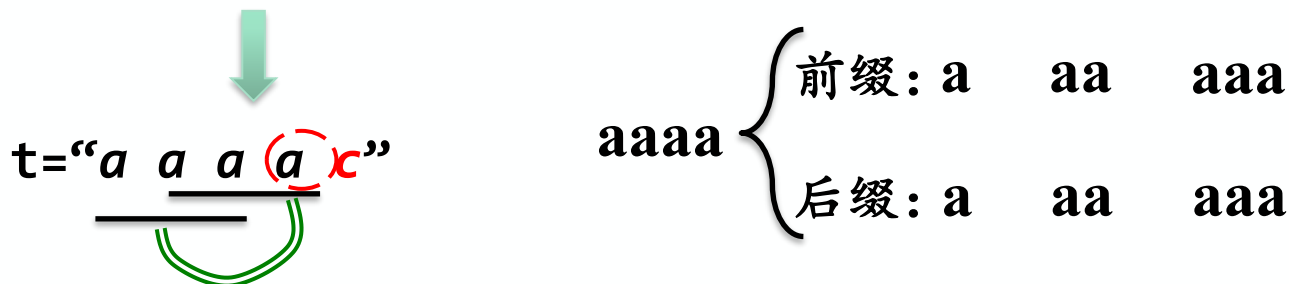
$$\underbrace{t_1 \cdots t_{k-1}}_{\text{开头的 } k-1 \text{ 个字符}} = \underbrace{t_{j-k+1} \cdots t_{j-1}}_{t_j \text{ 前面的 } k-1 \text{ 个字符}$$

开头的 $k-1$ 个字符 t_j 前面的 $k-1$ 个字符

求最大的 k

1 2 3 4 5

例如, $t = \text{"a a a a c"}$ 考虑 $t[5] = \text{'c'}$



⇒ $k=4$, 所以 $\text{next}[5] = k = 4$ 。

相等前后缀一定要尽可能地长, 即求满足条件的最大的 k , 这也使得模式串向右滑动尽可能远的距离。

$t = "aaab"$ 对应的 next 数组如下：

j	1	2	3	4
$t[j]$	a	a	a	b
$next[j]$	0	1	2	3

$$t_1 = t_2 = 'a' \quad t_1 t_2 = t_2 t_3 = "aa"$$

4.3.2 串的模式匹配算法——KMP算法

【例1】 写出串“ababa”的next数组

j	1	2	3	4	5
模式串	a	b	a	b	a
next[j]	0	1	1	2	3

next[j]如何计算?

(1)第1位为0, 第2位为1

(2)第 n 位($n \geq 3$)比较前 $n-1$ 位, 得出最大的前后缀匹配长度 k , 下一次匹配位置在 $k+1$, 填写 $k+1$

4.3.2 串的模式匹配算法——KMP算法

【例2】 写出串“abaabcac”的next数组

j	1	2	3	4	5	6	7	8
模式串	a	b	a	a	b	c	a	c
next[j]	0	1	1	2	2	3	1	2

next[j]如何计算?

(1)第1位为0, 第2位为1

(2)第 n 位($n \geq 3$)比较前 $n-1$ 位, 得出最大的前后缀匹配长度 k , 下一次匹配位置在 $k+1$, 填写 $k+1$

4.3.2 串的模式匹配算法——KMP算法

串“abcaabbcabcaabdab”的next数组

j	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
模式串	a	b	c	a	a	b	b	c	a	b	c	a	a	b	d	a	b
next[j]	0	1	1	1	2	2	3	1	1	2	3	4	5	6	7	1	2

next[j]如何计算?

(1)第1位为0, 第2位为1

(2)第n位($n \geq 3$)比较前n-1位, 得出最大的前后缀匹配长度k, 下一次匹配位置在k+1, 填写k+1

串“ababaaababaa”的next数组为（ ）？

A. -1, 0, 1, 2, 3, 4, 5, 6, 7, 8, 8, 8

B. -1, 0, 1, 0, 1, 0, 0, 0, 0, 1, 0, 1

C. -1, 0, 0, 1, 2, 3, 1, 1, 2, 3, 4, 5

D. -1, 0, 1, 2, -1, 0, 1, 2, 1, 1, 2, 3

答案：C

j	1	2	3	4	5	6	7	8	9	10	11	12
串	a	b	a	b	a	a	a	b	a	b	a	a
next[j]	0	1	1	2	3	4	2	2	3	4	5	6

解析：注意下标是从-1开始的，原本的next数组值减1。

4.3.2 串的模式匹配算法——KMP算法

另一种手工求解next数组的方法：

(1) $\text{next}[1]=0$, $\text{next}[2]=1$;

(2) 后面求解每一位的 $\text{next}[j]$ 的值时，与j的前一位进行比较，

先令 $k=\text{next}[j-1]$;

(3) 将 $S[j-1]$ 与 $S[k]$ 进行比较：

a. 若相等，则该 $\text{next}[j]=k+1$;

b. 若不相等，令 $k=\text{next}[k]$ ，若k不等于0，跳到(3);

若k等于0， $\text{next}[j]=1$ 。

4.3.2 串的模式匹配算法——KMP算法

- (1) $\text{next}[1]=0, \text{next}[2]=1$;
- (2) 后面求解每一位的 $\text{next}[j]$ 的值时, 与 j 的前一位进行比较, 先令 $k=\text{next}[j-1]$;
- (3) 将 $S[j-1]$ 与 $S[k]$ 进行比较:
 - a. 若相等, 则该 $\text{next}[j]=k+1$;
 - b. 若不相等, 令 $k=\text{next}[k]$, 若 k 不等于0, 跳到(3); 若 k 等于0, $\text{next}[j]=1$ 。

例如模式串 $S=\text{"abaabcac"}$, 依照以上算法求 next 数组。 $\text{next}[j]$

$\text{next}[1]=0, \text{next}[2]=1$ $j=3$ $\text{next}[3]=?$ 根据(2), $k=\text{next}[j-1]=\text{next}[2]=1$

(3) $S[3-1]$ 与 $S[1]$ 比较: 不等, $k=\text{next}[k]=\text{next}[1]=0$ $\text{next}[3]=1$

当 $j=4$ 根据(2), $k=\text{next}[4-1]=\text{next}[3]=1$

(3) $S[4-1]$ 与 $S[1]$ 比较: 相等, $\text{next}[4]=k+1=2$

当 $j=5$ 根据(2), $k=\text{next}[5-1]=\text{next}[4]=2$

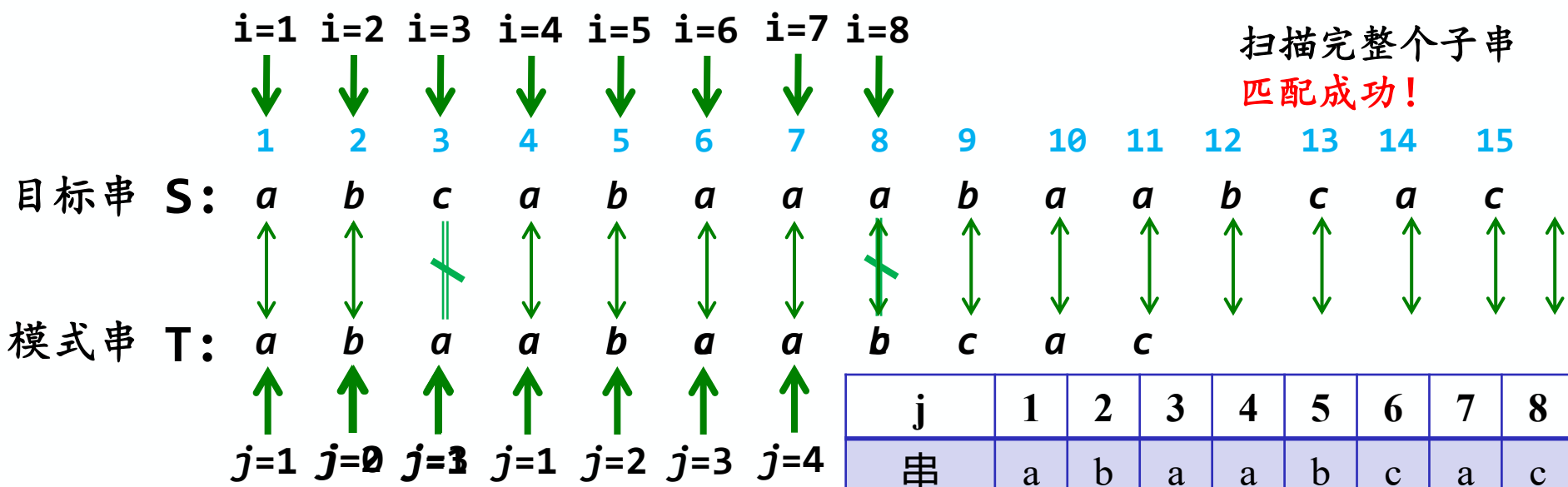
(3) $S[5-1]$ 与 $S[2]$ 比较: 不等, $k=\text{next}[k]=\text{next}[2]=1 \neq 0$, 跳到(3)

(3) $S[5-1]$ 与 $S[1]$ 比较: 相等, $\text{next}[5]=k+1=2$

4.3.2 串的模式匹配算法——KMP算法 (P81&82)

KMP算法的匹配过程:

假设主串S="abcabaaabaabcac", 子串T="abaabcac"



S[1]==T[1], 匹配: i和j自增1
S[2]==T[2], 匹配: i和j自增1
S[3]!=T[3], 不匹配: i不动, j=next[j]=1
S[3]!=T[1], 不匹配: i不动, j=next[j]=0
j=0, i和j自增1
S[4]==T[1], 匹配: i和j自增1
S[5]==T[2], 匹配: i和j自增1

S[6]==T[3], 匹配: i和j自增1
S[7]==T[4], 匹配: i和j自增1
S[8]!=T[5], 不匹配: i不变, j=next[j]=2
S[8]!=T[2], 不匹配: i不变, j=next[j]=1
S[8]==T[1], 匹配: i和j自增1 ...

扫描整个子串
匹配成功!

4.3.2 串的模式匹配算法——KMP算法

KMP算法一般步骤:

- (1) 设目标串(主串)为 S ，模式串为 T ，并设 i 和 j 的初值均为1。
- (2) 匹配过程中 若 $S_i == T_j$ ，则 i 和 j 分别自增1。
- (3) 否则若 $S_i \neq T_j$ ， i 不变， j 后退到 $next[j]$ 的位置，再比较 S_i 和 T_j ，依此类推。
- (4) 若当 j 退回到 $j = next[j] = 0$ ，将 i 和 j 分别自增1，重新开始匹配。
- (5) 结果：若子串扫描完毕（成功），则返回序号 $i - T[0]$ ；若主串全比较完毕都没有（失败），则返回0。

4.3.2 串的模式匹配算法——KMP算法 P116

```
int Index_KMP (SString S,SString T,int pos)
```

```
{ //利用next函数求子串T在主串S中第pos个字符之后失配、退回的位置
```

```
    i=pos, j=1;                //一般i=1,j=1
```

```
    while ( i<=S.ch[0] && j<=T.ch[0] ) //匹配过程中
```

```
    {
```

```
        if ( j==0 || S.ch[i]==T.ch[j] )        { ++i; ++j; }
```

```
        else
```

```
            j=next[j];                //i不变, j随着next数组后退
```

```
    }
```

```
    if (j>T.ch[0])    return i-T.ch[0];    //匹配成功,返回定位
```

```
    else                return 0;        //匹配失败,返回0
```

```
}
```

说明：本题为2015年全国考研题

已知正文串s为“*abaabaabacacaabaabcc*”，模式串t为“*abaabc*”，采用KMP算法进行匹配，第一次出现“失配” ($s[i] \neq t[j]$) 时， $i=j=5$ ，则下次开始匹配时， i 和 j 的值分别是（ ）。

A. $i=1, j=0$

B. $i=5, j=0$

C. $i=5, j=2$

D. $i=6, j=2$

i	1	2	3	4	5	6	9	10	11	12	13	14	15	16	17
$s[i]$	<i>a</i>	<i>b</i>	<i>a</i>	<i>a</i>	<i>b</i>	<i>a</i>	<i>a</i>	<i>b</i>	<i>a</i>	<i>c</i>	<i>a</i>	<i>c</i>	<i>a</i>	<i>a</i>	<i>b</i>
j	1	2	3	4	5	6									
$t[j]$	<i>a</i>	<i>b</i>	<i>a</i>	<i>a</i>	<i>b</i>	<i>c</i>									
next	0	1	1	2	2	3									

j	0	1	2	3	4	5
$t[j]$	<i>a</i>	<i>b</i>	<i>a</i>	<i>a</i>	<i>b</i>	<i>c</i>
next[j]	-1	0	0	1	1	2

选C

KMP算法的特点是在模式匹配时指示主串的指针（ ）

- A. 不会变大**
- B. 不会变小**
- C. 都有可能**
- D. 无法判断**

答案： B

解释： P92&94， KMP算法中， 失配时主串指针不会回溯

示例

设主串 $s = \text{"aaabaaaaab"}$ ，子串 $t = \text{"aaaaab"}$ 。

KMP模式匹配过程：

(1) 求子串 t 的 next 数组

j	1	2	3	4	5
$t[j]$	<i>a</i>	<i>a</i>	<i>a</i>	<i>a</i>	<i>b</i>
$\text{next}[j]$	0	1	2	3	4

j	1	2	3	4	5
$t[j]$	<i>a</i>	<i>a</i>	<i>a</i>	<i>a</i>	<i>b</i>
$next[j]$	0	1	2	3	4

	1	2	3	4	5	6	7	8	9
$s:$	<i>a</i>	<i>a</i>	<i>a</i>	<i>b</i>	<i>a</i>	<i>a</i>	<i>a</i>	<i>a</i>	<i>b</i>
$t:$	<i>a</i>	<i>a</i>	<i>a</i>	<i>a</i>	<i>b</i>				
	1	2	3	4	5				

Diagram illustrating the KMP algorithm's failure function calculation. The string s is "a a a b a a a a b" and the string t is "a a a a b". The failure function values are shown below t . The mismatch at index 4 is highlighted with a pink double line and a diagonal slash.

失败:

$i=4$

$j=next[4]=3$

j	1	2	3	4	5
$t[j]$	<i>a</i>	<i>a</i>	<i>a</i>	<i>a</i>	<i>b</i>
$next[j]$	0	1	2	3	4

		1	2	3	4	5	6	7	8	9
$i=4$	$S:$	<i>a</i>	<i>a</i>	<i>a</i>	<i>b</i>	<i>a</i>	<i>a</i>	<i>a</i>	<i>a</i>	<i>b</i>
$j=3$	$t:$	<i>a</i>	<i>a</i>	<i>a</i>	<i>a</i>	<i>b</i>				
		1	2	3	4	5				

失败:

$i=4$

$j=next[3]=2$

j	1	2	3	4	5
t[j]	a	a	a	a	b
next[j]	0	1	2	3	4

		1	2	3	4	5	6	7	8	9
i=4	S:	a	a	a	b	a	a	a	a	b
j=2	t:		a	a	a	a	b			
		1	2	3	4	5				

失败:

i=4

j=next[2]=1

j	1	2	3	4	5
$t[j]$	<i>a</i>	<i>a</i>	<i>a</i>	<i>a</i>	<i>b</i>
$next[j]$	0	1	2	3	4

		1	2	3	4	5	6	7	8	9
$i=4$	$S:$	<i>a</i>	<i>a</i>	<i>a</i>	<i>b</i>	<i>a</i>	<i>a</i>	<i>a</i>	<i>a</i>	<i>b</i>
$j=1$	$t:$		<i>a</i>	<i>a</i>	<i>a</i>	<i>a</i>	<i>b</i>			
		1	2	3	4	5				

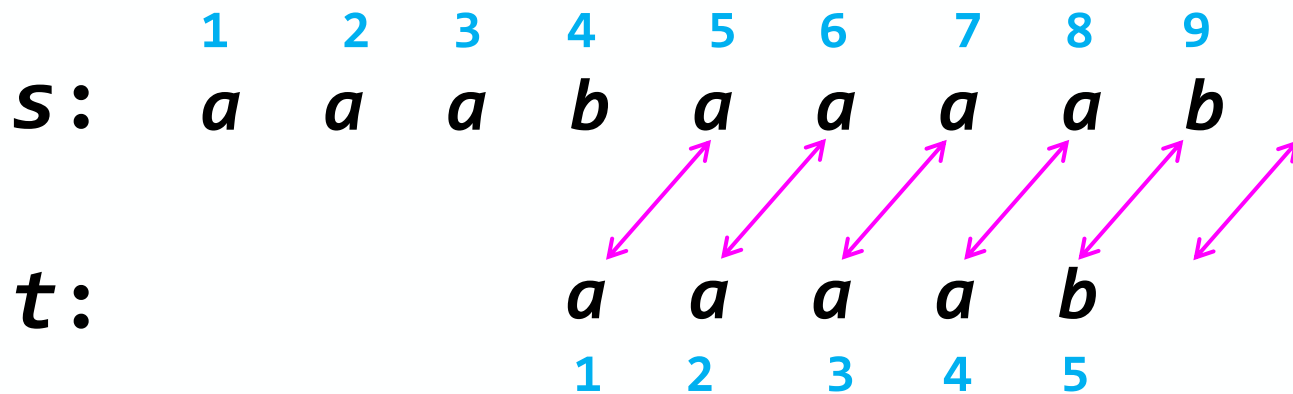
失败:

$i=4$

$j=next[1]=0$

j	1	2	3	4	5
t[j]	a	a	a	a	b
next[j]	0	1	2	3	4

因为 $j=0$:
 $i++$;
 $j++$;
 即:
 $i=5$
 $j=1$

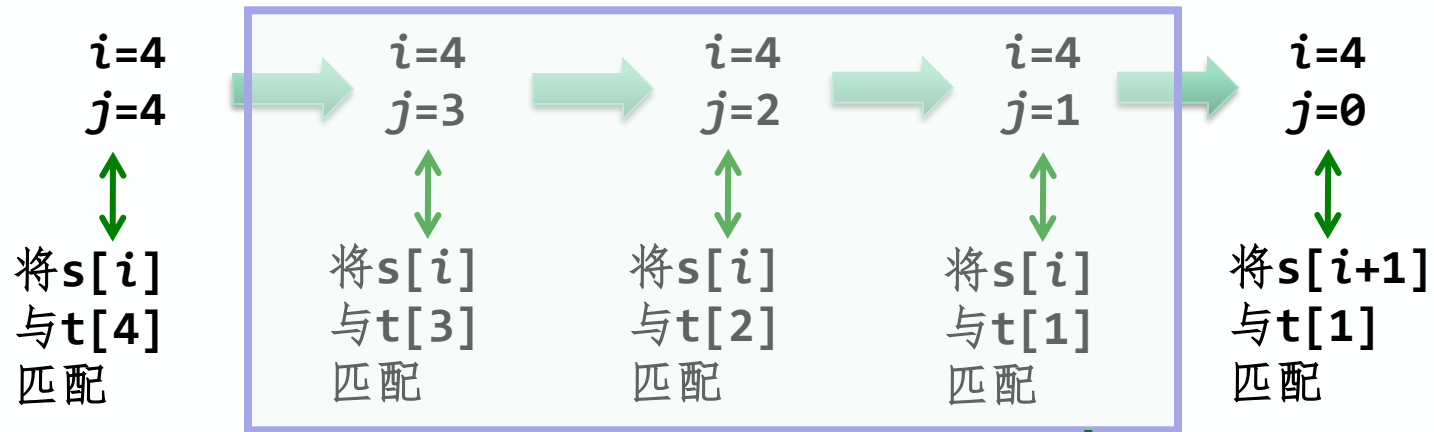


成功匹配:

返回 $i - t.length = 10 - 5 = 5$

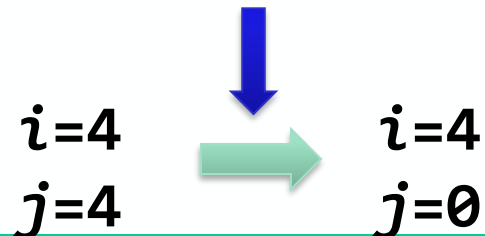
j	1	2	3	4	5
t[j]	a	a	a	a	b
next[j]	0	1	2	3	4

前面的匹配过程:



是不必要的

因为 $t[4]=t[3]=t[2]=t[1]='a'$



将next改为nextval:

j	1	2	3	4	5
t[j]	a	a	a	a	b
next[j]	0	1	2	3	4
nextval[j]	0	0	0	0	4

next[2]=1
t[2]==t[next[2]]=t[1]='a'
∴ nextval[2]=nextval[1]=0

t[5]='b' ≠ t[next[5]]='a'
∴ nextval[5]=next[5]



- nextval[1]=0
- 当t[j]==t[next[j]]时: nextval[j]=nextval[next[j]]
- 否则: nextval[j]=next[j]

用nextval取代next, 得到改进的KMP算法。

使用改进后的KMP算法示例：

j	1	2	3	4	5
t[j]	a	a	a	a	b
nextval[j]	0	0	0	0	4

	1	2	3	4	5	6	7	8	9
S:	a	a	a	b	a	a	a	a	b
	↑↓	↑↓	↑↓						
t:	a	a	a	a	b				
	1	2	3	4	5				

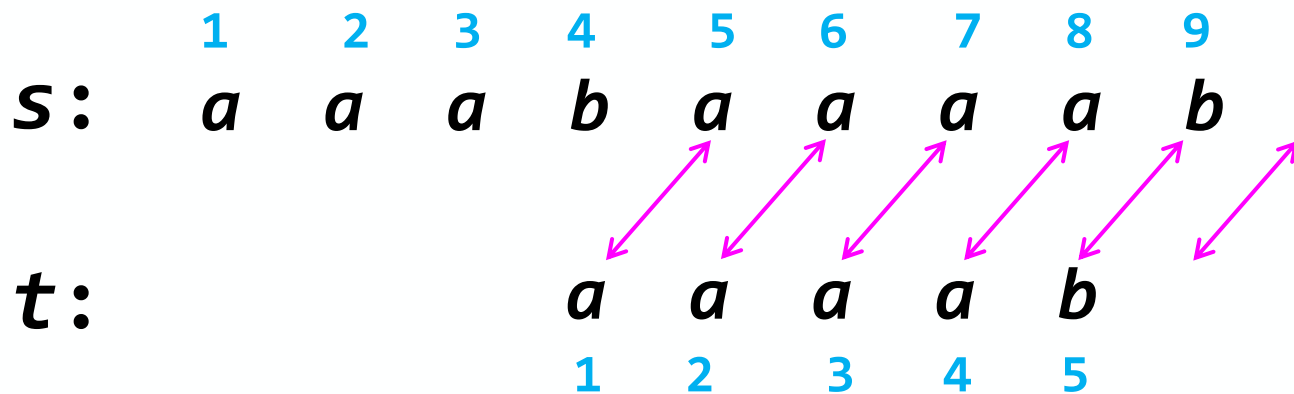
失败:

$i=4$

$j=\text{nextval}[4]=0$

j	1	2	3	4	5
t[j]	a	a	a	a	b
nextval[j]	0	0	0	0	4

因为 $j=0$:
 $i++$;
 $j++$;
 即:
 $i=5$
 $j=1$



成功匹配:

返回 $i - t.length$
 $= 10 - 5 = 5$



改进后的KMP算法进一步提高模式匹配的效率。

根据next值求nextval值的方法：

j	1	2	3	4	5	6	7	8
模式串T[j]	a	b	a	a	b	c	a	c
next[j]	0	1	1	2	2	3	1	2
nextval[j]	0	1	0	2	1	3	0	2

1. 第一位的nextval值必定为0，第二位如果与第一位相同则为0，如果不同则为next[2]。
2. 第三位的next值为1，那么将第三位和第一位进行比较，均为a，相同，则第三位的nextval值为第一位的nextval值，为0。
3. 第四位的next值为2，那么将第四位和第二位进行比较，不同，则第四位的nextval值为其next值，为2。
4. 第五位的next值为2，那么将第五位和第二位进行比较，相同，第五位的nextval值为第二位的nextval值，为1。
5. 第六位的next值为3，那么将第六位和第三位进行比较，不同，则第六位的nextval值为其next值，为3。
6. 第七位的next值为1，那么将第七位和第一位进行比较，相同，则第七位的nextval值为0。
7. 第八位的next值为2，那么将第八位和第二位进行比较，不同，则第八位的nextval值为其next值，为2。

next函数修正值——nextval函数

【例5】 已知模式串 $t = \text{"ABABAAB"}$

写出用KMP算法对应的next和nextval函数值。

【解析】

模式串 t 的next和nextval值如下：

j	1	2	3	4	5	6	7
模式串	A	B	A	B	A	A	B
next[j]	0	1	1	2	3	4	2
nextval[j]	0	1	0	1	0	4	1

【例6】串“ababaaba”的nextval值

j	1	2	3	4	5	6	7	8
模式串	a	b	a	b	a	a	b	a
next[j]	0	1	1	2	3	4	2	3
nextval[j]	0	1	0	1	0	4	1	0

【例7】 串“abcaabbabcbab”的nextval函数值

j	1	2	3	4	5	6	7	8	9	10	11	12
模式串	a	b	c	a	a	b	b	a	b	c	a	b
next[j]	0	1	1	1	2	2	3	1	2	3	4	5
nextval[j]	0	1	1	0	2	1	3	0	1	1	0	5

练习一下

1. 串的长度是指 (B)

- A. 串中所含不同字母的个数 B. 串中所含字符的个数
C. 串中所含不同字符的个数 D. 串中所含非空格字符的个数

2. 为查找某一特定单词在文本中出现的位置，可应用的串运算是 (D)

- A. 插入 B. 删除 C. 串连接 D. 子串定位

3. 设有两个串p和q，求q在p中首次出现的位置的运算称为 (B)

- A. 连接串 B. 模式匹配 C. 复制串 D. 求串长

4.4 数组的定义 P118

从逻辑结构上看，数组是 n ($n > 0$) 个按一定格式排列起来的、具有相同类型的数据元素 a_1 、 a_2 、...、 a_n 构成的有限序列。

其中，一维数组：若线性表中的数据元素为非结构的简单元素，则称为一维数组。

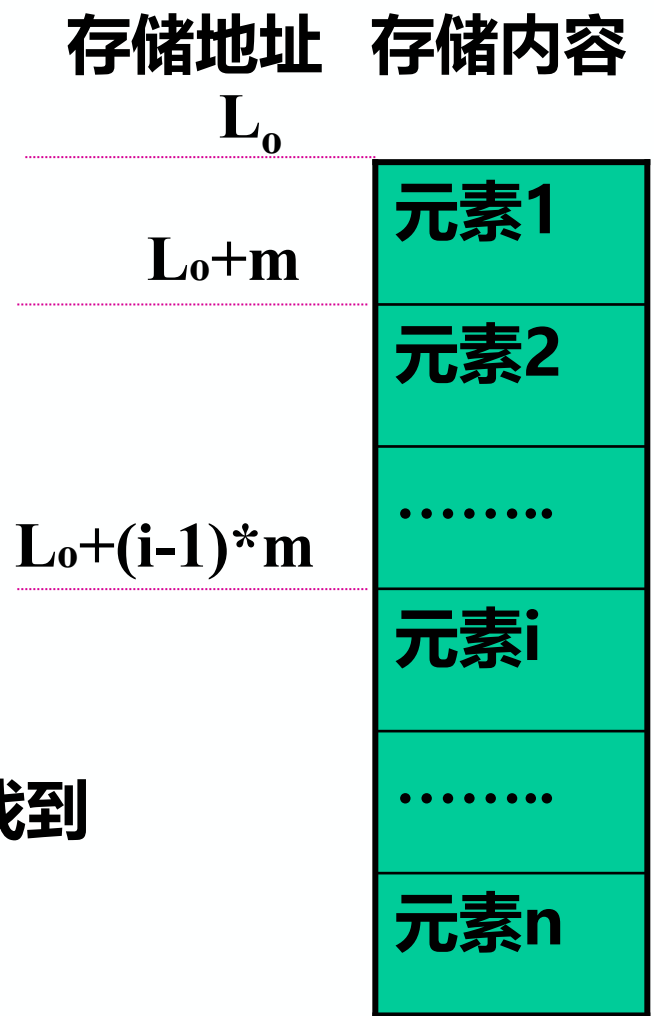
其逻辑表示为：

$$A = (a_1, a_2, \dots, a_n)$$

其中， a_i ($1 \leq i \leq n$) 表示数组 A 的第 i 个元素。

4.4.1 一维数组的顺序存储

L_0 为首元素地址，
 m 为每个数据元素的长度



随机存取特性：
可以在 $O(1)$ 时间内找到
序号为 i 的元素值。

2000	a[0]
2001	
2002	
2003	
2004	a[1]
2005	
2006	
2007	
2008	a[2]
2009	
2010	
2011	
2012	a[3]
2013	
2014	
2015	
2016	a[4]
2017	
2018	
2019	

每个元素占用4字节，假设 $a[0]$ 存储在2000单元， $a[3]$ 在哪？
 $2000+3*4$

4.4.2 二维数组

例： `int a[3][3];`

<code>a[0]</code>	<code>a[0][0]</code>	<code>a[0][1]</code>	<code>a[0][2]</code>
<code>a[1]</code>	<code>a[1][0]</code>	<code>a[1][1]</code>	<code>a[1][2]</code>
<code>a[2]</code>	<code>a[2][0]</code>	<code>a[2][1]</code>	<code>a[2][2]</code>

多维数组是线性表的推广，一个n维数组实质上是n个线性表的组合。

4.4.2 二维数组的顺序存储 P119

m 行 n 列的二维数组 $A_{m \times n}$,

存储方式:

- 以行序为主序的存储
- 以列序为主序的存储

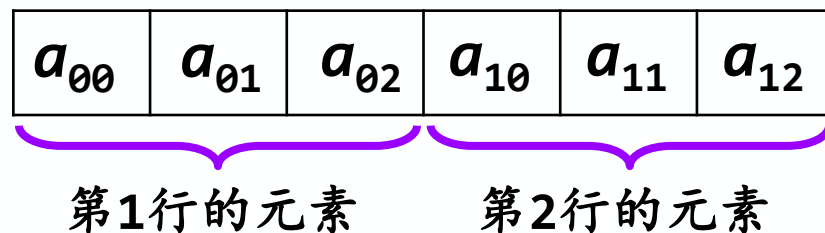
$$A = \begin{bmatrix} a_{00} & a_{01} & \cdots & a_{0,n-1} \\ a_{10} & a_{11} & \cdots & a_{1,n-1} \\ \cdots & \cdots & \cdots & \cdots \\ a_{m-1,0} & a_{m-1,1} & \cdots & a_{m-1,n-1} \end{bmatrix}$$

$$a_j = (a_{0j}, a_{1j}, a_{2j}, \dots, a_{m-1,j}) \quad 0 \leq j \leq n-1$$

$$a_i = (a_{i0}, a_{i1}, a_{i2}, \dots, a_{i,n-1}) \quad 0 \leq i \leq m-1$$

① 以行序为主序的存储方式

$$A = \begin{bmatrix} a_{00} & a_{01} & a_{02} \\ a_{10} & a_{11} & a_{12} \end{bmatrix}$$



• 以行序为主序

C、PASCAL

$$A = \begin{bmatrix} a_{00} & a_{01} & \cdots & a_{0,n-1} \\ a_{10} & a_{11} & \cdots & a_{1,n-1} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m-1,0} & a_{m-1,1} & \cdots & a_{m-1,n-1} \end{bmatrix}$$

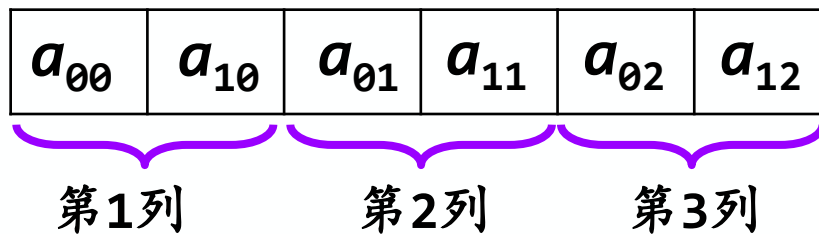
$a_{00}, a_{01}, \dots, a_{0,n-1}$
第1行的元素

$a_{10}, a_{11}, \dots, a_{1,n-1}$
第2行的元素

$a_{m-1,0}, a_{m-1,1}, \dots, a_{m-1,n-1}$
第m行的元素

② 以列序为主序的存储方式

$$A = \begin{bmatrix} a_{00} & a_{01} & a_{02} \\ a_{10} & a_{11} & a_{12} \end{bmatrix}$$



• 以列序为主序
FORTRAN

$$A = \begin{bmatrix} a_{00} & a_{01} & \cdots & a_{0,n-1} \\ a_{10} & a_{11} & \cdots & a_{1,n-1} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m-1,0} & a_{m-1,1} & \cdots & a_{m-1,n-1} \end{bmatrix}$$

a_{00}
 a_{10}
...
 $a_{m-1,0}$

第1列的元素

a_{01}
 a_{11}
...
 $a_{m-1,1}$

第2列的元素

$a_{0,n-1}$
 $a_{1,n-1}$
...
 $a_{m-1,n-1}$

第n列的元素

4.4.2 二维数组的**行序**优先表示

$a[0\dots m-1][0\dots n-1]$: 下标从0开始, 共有m行n列

$$a = \begin{pmatrix} a[0][0] & a[0][1] & \dots & a[0][n-1] \\ a[1][0] & a[1][1] & \dots & a[1][n-1] \\ a[2][0] & a[2][1] & \dots & a[2][n-1] \\ \dots & \dots & \dots & \dots \\ a[m-1][0] & a[m-1][1] & \dots & a[m-1][n-1] \end{pmatrix}$$

若每个数据元素占L个存储单元, $a[i][j]$ 存放位置

$$\text{LOC}(i, j) = \text{LOC}(0, 0) + (i*n + j)L$$

$a[i][j]$ 前面的所有元素个数

二维数组采用顺序存储结构时, 也具有**随机存取特性**。

是指给定序号 (i, j) , 可以在 $O(1)$ 的时间内找到相应的元素值。

4.4.2 二维数组的列序优先表示

$a[0\dots m-1][0\dots n-1]$: 下标从0开始, 共有m行n列

$$a = \begin{pmatrix} a[0][0] & a[0][1] & \dots & a[0][n-1] \\ a[1][0] & a[1][1] & \dots & a[1][n-1] \\ a[2][0] & a[2][1] & \dots & a[2][n-1] \\ \dots & \dots & \dots & \dots \\ a[m-1][0] & a[m-1][1] & \dots & a[m-1][n-1] \end{pmatrix}$$

若每个数据元素占L个存储单元, $a[i][j]$ 存放位置

$$\text{LOC}(i, j) = \text{LOC}(0, 0) + (j*m + i)L$$

$a[i][j]$ 前面的所有元素个数

二维数组采用顺序存储结构时, 也具有随机存取特性。

是指给定序号 (i, j) , 可以在 $O(1)$ 的时间内找到相应的元素值。

练习1

设有二维数组A[10,20]，其每个元素占两个字节，A[0][0]存储地址为100，若按行优先顺序存储，则元素A[6,7]的存储地址为 **354** 按列优先顺序存储，元素A[6,7]的存储地址为 **252**

$$\begin{aligned}\text{LOC}(6,7) &= \text{LOC}(0,0) + (i * n + j) * L \\ &= 100 + (6 * 20 + 7) * 2 \\ \text{行优先} &= 354\end{aligned}$$

$$\begin{aligned}\text{LOC}(6,7) &= \text{LOC}(0,0) + (j * m + i) * L \\ &= 100 + (10 * 7 + 6) * 2 \\ \text{列优先} &= 252\end{aligned}$$

【练习2】假设以**行序**为主序存储二维数组

array[**1**..100,**1**..90]，设每个数据元素占2个存储单元，
基地址为18，则LOC[6,5]=（ ）。

A. 808

B. 926

C. 1010

D. 1562

答案：B

解释：以行序为主，

则 $LOC[6,5]=18+[(6-1)*90+(5-1)]*2=926$

$$a = \begin{pmatrix} a[1][1] & a[1][2] & \dots & a[1][90] \\ a[2][1] & a[2][2] & \dots & a[2][90] \\ a[3][1] & a[3][2] & \dots & a[3][90] \\ \dots & \dots & \dots & \dots \\ a[100][1] & a[100][2] & \dots & a[100][90] \end{pmatrix}$$

$a[6][5]$

【练习3】 设有数组A[m][n]按**行**优先顺序存储，
A[0][0]存放位置在644，A[2][2]存放位置在676，
每个元素占一个空间，问元素A[3][3]的存储地址
为（ ）。

A. 682

B. 689

C. 692

D. 699

答案：C

解释：以行序为主，则

$$\text{LOC}[2, 2] = [2 * n + 2] * 1 + 644 = 676 \quad n = 15$$

$$\text{LOC}[3, 3] = [3 * n + 3] * 1 + 644 = (3 * 15 + 3) + 644 = 692$$

【练习4】 在一个二维数组A中，假设每个数组元素的长度为3个存储单元，行下标i为0~8，列下标j为1~9，从首地址SA开始连续存放。当以行序为主，元素A[8][5]的起始地址为（ ）。

A. SA+141

B. SA+144

C. SA+228

D. SA+255

答案：C

解释：以行序为主，则

$$\text{LOC}[8, 5] = [8 \times 9 + (5 - 1)] \times 3 + \text{SA} = \text{SA} + 228$$

【练习5】 数组A[0..4,-1..-3,5..7]中含有元素的个数 () 。

A. 55

B. 45

C. 36

D. 16

答案： B

解释： 共有 $(4-0+1)*(-1-(-3)+1)*(7-5+1) = 45$ 个元素。 $5*3*3=45$ 个

4.4.3 特殊矩阵的压缩存储 (P120)

在高级语言编程时，通常将一个矩阵描述为一个二维数组，可以对其元素进行随机存取，各种矩阵运算也非常简单，存储密度为1。

对于高阶矩阵，若其中**非零元素呈某种规律分布或者矩阵中有大量的零元素**，若仍然用常规方法存储，会造成存储重复，存储空间的大量浪费。

对这类矩阵（对称矩阵，对角矩阵，稀疏矩阵等）进行压缩存储：

- ◆ 多个**相同的非零元素**只分配一个存储空间；
- ◆ **零元素**不分配空间。

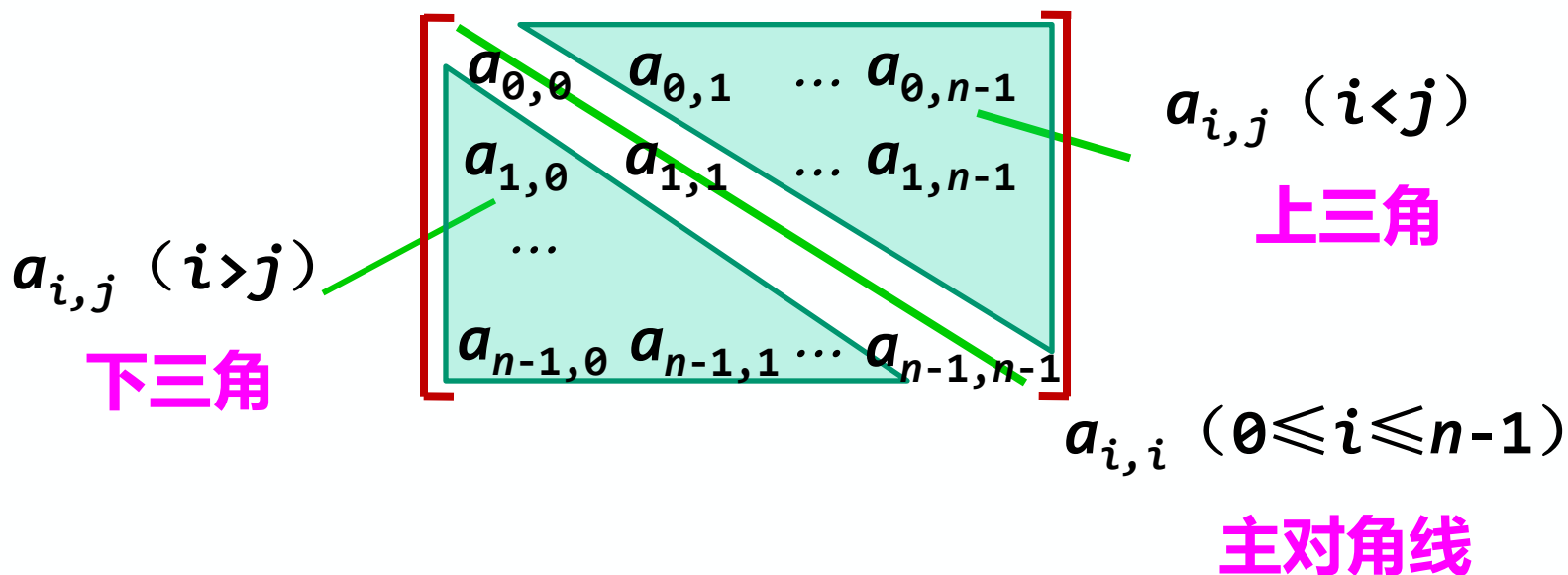


节省存储空间

对称矩阵(P121)

【定义】 若一个n阶方阵 $A=(a_{ij})_{n \times n}$ 中的元素满足性质：

$$a_{ij}=a_{ji} \quad 0 \leq i, j \leq n-1 \text{ 且 } i \neq j \quad \text{则称} A \text{ 为} n \text{ 阶对称矩阵。}$$



对称矩阵如何实现压缩存储？

对称矩阵(P121)

第一行1个元素
第二行2个元素
A=
第n行n个元素

$$A = \begin{pmatrix} 1 & 5 & 1 & 3 & 7 \\ 5 & 0 & 8 & 0 & 0 \\ 1 & 8 & 9 & 2 & 6 \\ 3 & 0 & 2 & 5 & 1 \\ 7 & 0 & 6 & 1 & 3 \end{pmatrix}$$

$$A = \begin{pmatrix} a_{11} & & & & \\ a_{21} & a_{22} & & & \\ a_{31} & a_{32} & a_{33} & & \\ \dots & \dots & \dots & \dots & \\ a_{n1} & a_{n2} & \dots & & a_{nn} \end{pmatrix}$$

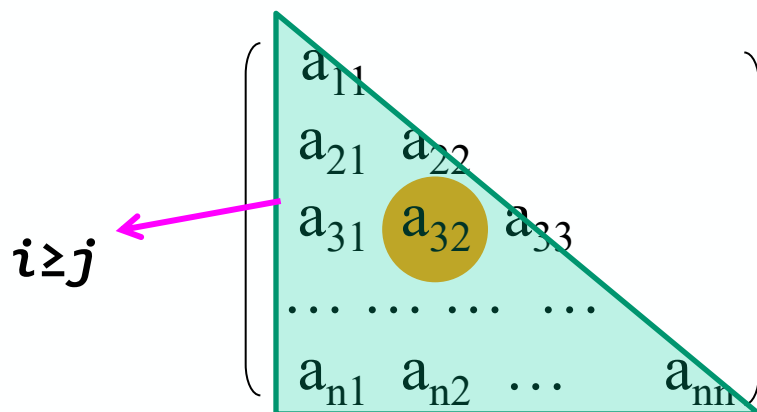
图 对称矩阵示例

[存储方法] 只存储下(或者上)三角、包括主对角线的数据元素
共占用 $n(n+1)/2$ 个元素空间。

$$\text{元素总数} = 1 + 2 + 3 + \dots + n = (1+n) * n / 2 = n(n+1) / 2$$

对称矩阵(P121)

以行序为主序存储其下三角+主对角线的元素。



下三角+主对角线

$a_{i,j}$



$k = ?$

$sa \rightarrow k$ 0 1 2 3 ...

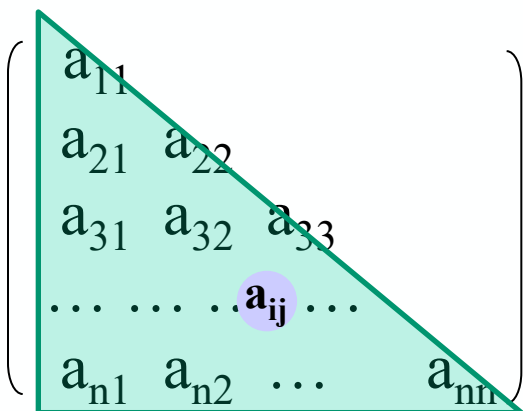
$n(n+1)/2 - 1$

a_{11}	a_{21}	a_{22}	a_{31}	a_{32}	a_{33}	$\dots$	a_{n1}	a_{n2}	$\dots$	a_{nn}
----------	----------	----------	----------	----------	----------	---------	----------	----------	---------	----------

$n(n+1)/2$ 个元素

对称矩阵(P121)

以行序为主序存储其下三角+主对角线的元素。



在数组sa中，位于元素 $a_{i,j}$ ($i \geq j$) 前面的元素：

第1行：1个元素 (a_{11})

第2行：2个元素 (a_{21}, a_{22})

第3行：3个元素 (a_{31}, a_{32}, a_{33})

.....

第i-1行：i-1个元素 ($a_{i-1,1}, a_{i-1,2}, \dots, a_{i-1,i-1}$)

第i行：j-1个元素 ($a_{i,1}, a_{i,2}, \dots, a_{i,j-1}$)

$a_{i,j}$ 前面的元素个数 $= (1+2+3+\dots+i-1) + j-1 = (1+i-1)*(i-1)/2 + j-1 = i(i-1)/2 + j-1$

sa $\rightarrow$ k 0 1 2 3 $n(n+1)/2-1$

a_{11}	a_{21}	a_{22}	a_{31}	a_{32}	a_{33}	...	a_{n1}	a_{n2}	...	a_{nn}
----------	----------	----------	----------	----------	----------	-----	----------	----------	-----	----------

对称矩阵(P121)

假设按 **“行优先顺序”** 存储**下三角形**(包括对角线)中的元素。

设用一维数组**sa[0...n(n+1)/2-1]**存储n阶对称矩阵，为了便于访问，矩阵A中元素的下标值(i, j)和**sa[k]**的下标值k之间的对应关系如下：

$$A = \begin{pmatrix} a_{11} & & & & \\ a_{21} & a_{22} & & & \\ a_{31} & a_{32} & a_{33} & & \\ \dots & \dots & \dots & \dots & \\ a_{n1} & a_{n2} & \dots & \dots & a_{nn} \end{pmatrix}$$

$$k = \begin{cases} i(i-1)/2 + j - 1 & \text{当 } i \geq j \\ j(j-1)/2 + i - 1 & \text{当 } i < j \end{cases}$$

【练习】 设有一个10阶的对称矩阵A，采用压缩存储方式，以行序为主存储， a_{11} 为第一个元素，其存储地址为1，每个元素占一个地址空间，则 a_{85} 的地址为（ ）。

A. 13

B. 32

C. 33

D. 40

答案：C

解析： a_{85} 的地址为

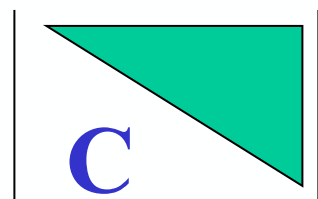
$$1+2+3+4+5+6+7+4+1=33$$

比如 a_{33} 地址为6，前面有5个元素，需要 $5*1+1$

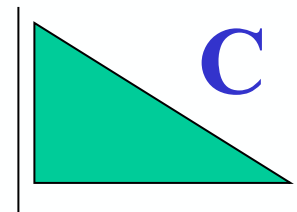
$$\begin{pmatrix} a_{11} & & & & \\ a_{21} & a_{22} & & & \\ a_{31} & a_{32} & a_{33} & & \\ \dots & \dots & \dots & \dots & \\ a_{n1} & a_{n2} & \dots & \dots & a_{nn} \end{pmatrix}$$

2. 三角矩阵(P121)

[特点] 对角线以下(或者以上)的数据元素(不包括对角线)全部为常数 c 。



上三角矩阵



下三角矩阵

[存储方法] 重复元素 c 共享一个元素存储空间，共占用 $n(n+1)/2+1$ 个元素空间： $sa[0.. n(n+1)/2]$

k	0	1	2	3	...	...						
	a_{11}	a_{21}	a_{22}	a_{31}	a_{32}	a_{33}	...	a_{n1}	a_{n2}	...	a_{nn}	c

下三角矩阵

$sa[k]$ 和矩阵 $a_{i,j}$ 之间的
对应关系

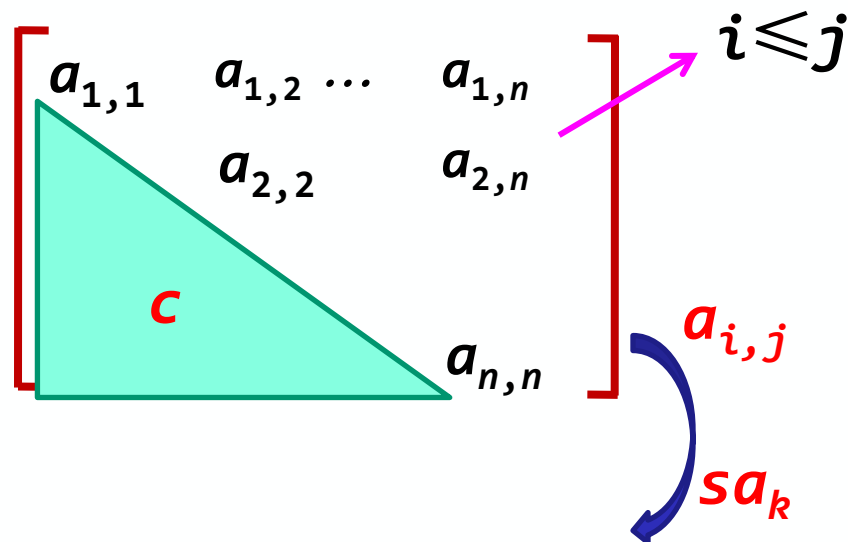
$$k = \begin{cases} i(i-1)/2 + j - 1 & \text{当 } i \geq j \\ n(n+1)/2 & \text{当 } i < j \end{cases}$$

存放一个常量 c

2. 三角矩阵(P121)



上三角矩阵



$$sa = (a_{1,1}, a_{1,2}, \dots, a_{1,n}, a_{2,2}, \dots, a_{2,n}, a_{3,3}, \dots, a_{n,n})$$

在数组sa中，位于元素 $a_{i,j}$ ($i \leq j$)前面的元素：

第1行： $n-1+1=n$ 个元素 ($a_{11}, a_{12}, \dots, a_{1,n}$)

第2行： $n-2+1=n-1$ 个元素 ($a_{22}, a_{23}, \dots, a_{2,n}$)

第3行： $n-3+1=n-2$ 个元素 ($a_{33}, \dots, a_{3,n}$)

.....

第 $i-1$ 行： $n-(i-1)+1=n-i+2$ 个元素 ($a_{i-1,i-1}, \dots, a_{i-1,n}$)

第 i 行： $j-i$ 个元素 ($a_{i,i}, a_{i,i+1}, \dots, a_{i,j-1}$)

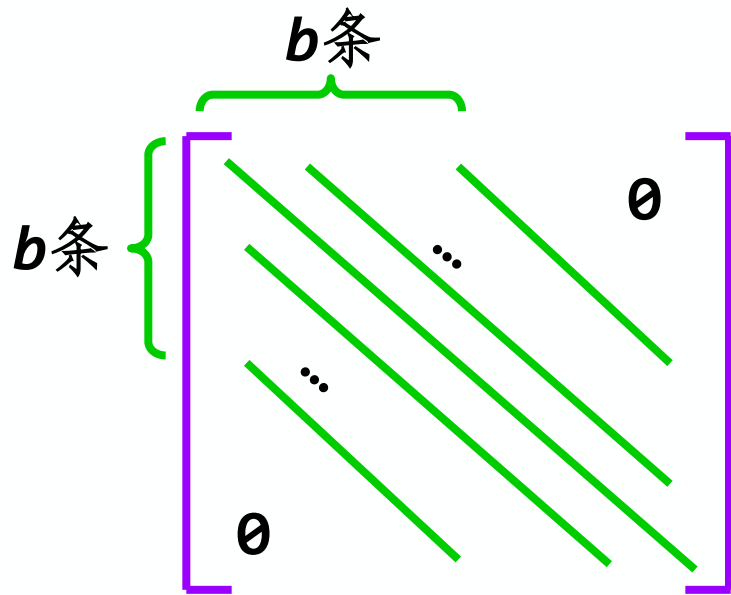
$a_{i,j}$ 前面的元素个数

$$=(n+n-1+\dots+n-i+2)+j-i$$

$$=(2n-i+2)*(i-1)/2+j-i$$

多对角线矩阵(P121)

【特点】 也称带状矩阵，在 $n \times n$ 的方阵中，所有非零元素都集中在以主对角线为中心的带状区域中，区域外的值全为0，也称为**对角矩阵**。常见的有三对角矩阵、五对角矩阵、七对角矩阵等



b 条对角线的对角矩阵

多对角线矩阵(P122)

【存储方法】 以五对角矩阵为例：

$$\begin{pmatrix} 8 & 2 & 3 & 0 & 0 & 0 \\ 4 & 2 & 0 & 3 & 0 & 0 \\ 5 & 7 & 7 & 6 & 8 & 0 \\ 0 & 9 & 6 & 9 & 1 & 5 \\ 0 & 0 & 6 & 1 & 4 & 2 \\ 0 & 0 & 0 & 2 & 8 & 3 \end{pmatrix}$$

6*6=36个存储空间

	1	2	3	4	5	6
-2			3	3	8	5
-1		2	0	6	1	2
0	8	2	7	9	4	3
1	4	7	6	1	8	
2	5	9	6	2		

5*6=30个存储空间

稀疏矩阵(P122)

【定义】 一个阶数较大的矩阵中的非零元素个数 s 相对于矩阵元素的总个数 t 十分小时，即 $s \ll t$ 时，称该矩阵为稀疏矩阵。

例如一个 100×100 的矩阵，若其中只有100个非零元素，就可称其为稀疏矩阵。（稀疏因子 ≤ 0.05 ） $100/10000=0.01$

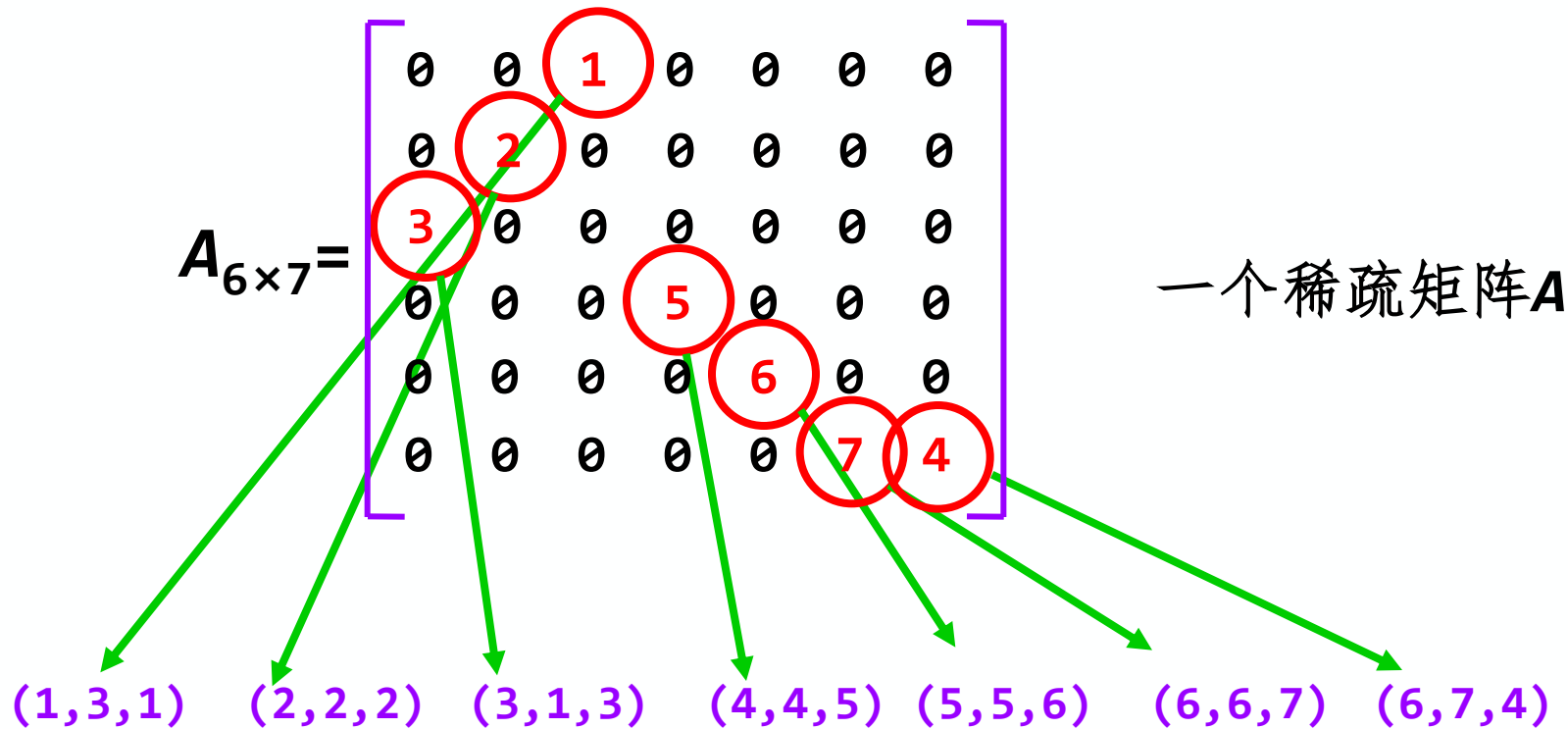
【三元组表示法】 稀疏矩阵的压缩存储方法是只存储非零元素。
稀疏矩阵中的每一个非零元素可以由一个三元组：

$$(i, j, a_{i,j})$$

唯一确定，稀疏矩阵中的所有非零元素构成三元组顺序表。

稀疏矩阵(P122)

一个 6×7 阶稀疏矩阵A的三元组顺序表表示



三元组顺序表:

$((1,3,1), (2,2,2), (3,1,3), (4,4,5), (5,5,6), (6,6,7), (6,7,4))$

稀疏矩阵(P122)

三元组顺序表：

	i	j	value
0	6	7	7
1	1	3	1
2	2	2	2
3	3	1	3
4	4	4	5
5	5	5	6
6	6	6	7
7	6	7	4

3*8=24个存储空间

6*7=42个存储空间

0	0	1	0	0	0	0
0	2	0	0	0	0	0
3	0	0	0	0	0	0
0	0	0	5	0	0	0
0	0	0	0	6	0	0
0	0	0	0	0	7	4

注意：为更可靠描述，通常再加一个“总体”信息：即总行数、总列数、非零元素总个数。

稀疏矩阵(P122)

下面的三元组顺序表示一个稀疏矩阵，试还原出它的稀疏矩阵。

	i	j	value
0	6	4	6
1	1	2	2
2	2	1	12
3	3	1	3
4	4	4	4
5	5	3	6
6	6	1	16

$$\begin{pmatrix} 0 & 2 & 0 & 0 \\ 12 & 0 & 0 & 0 \\ 3 & 0 & 0 & 0 \\ 0 & 0 & 0 & 4 \\ 0 & 0 & 6 & 0 \\ 16 & 0 & 0 & 0 \end{pmatrix}$$

【练习】 有一个 100×90 的稀疏矩阵，非0元素有10个，用三元组表示该矩阵，设三元组顺序表中每个整型常量占2字节，则所需的字节数是（ ）。

A. 60

B. 66

C. 18000

D. 33

答案：B

解析：

	i	j	value
0	100	90	10
1			
2			
3			
...	...	...	...
10			

$11 \times 3 \times 2 = 66$ 个字节

练习一下

1. 一个 $n \times n$ 的对称矩阵，若以行或列为主序放入内存，则容量为（ C ）。

A. $n \times n$ B. $n \times n / 2$ C. $n \times (n+1) / 2$ D. $(n+1)^2 / 2$

2. 对稀疏矩阵进行压缩存储目的是（ C ）。

A. 便于进行矩阵运算 B. 便于输入和输出
C. 节省存储空间 D. 降低运算的时间复杂度

4.5 广义表(P122)

广义表是线性表的推广和扩充，也称为列表(Lists)。

在第2章中，我们把线性表定义为 $n(n \geq 0)$ 个元素 $a_1, a_2, \dots, a_n$ 的有穷序列，该序列中的所有元素具有相同的数据类型且只能是原子项。所谓**原子项是指结构上不可再分的**。

若放松对元素的这种限制，容许它们具有其自身结构，就产生了广义表。如： $C = (a, (b, c, d))$

4.5.1 广义表(P122)

➤ **广义表**通常记作： $LS=(a_1, a_2, \dots, a_n)$

其中： LS 为**表名**， n 为**表的长度**，每一个 a_i 为**表的元素**。

➤ 习惯上，用**大写字母**表示**广义表**，**小写字母**表示**原子**。

➤ **表头**：若 LS 非空($n \geq 1$)，则其第一个元素 a_1 就是表头。

记作 $\text{GetHead}(LS) = a_1$ 注：表头可以是原子，也可以是子表。

➤ **表尾**：除表头之外的**其它元素组成的表**。

记作 $\text{GetTail}(LS) = (a_2, \dots, a_n)$

注：表尾不是最后一个元素，而是一个子表。

【练习1】 广义表((a,b,c,d))的表头是 () , 表尾是 () 。

A. a

B. (a,b,c,d)

C. ()

D. (b,c,d)

答案： B、 C

解释：表头为非空广义表的第一个元素，可以是一个单原子，也可以是一个子表，((a,b,c,d))的表头为一个子表(a,b,c,d)。表尾为除去表头之外，由其余元素构成的表，((a,b,c,d))的表尾为空表()。

4.5.1 广义表(P122)

广义表的长度和深度:

- 广义表的**长度**定义为最外层包含元素个数。
- 广义表的**深度**定义为所含括弧的重数。其中，原子的深度为0，空表的深度为1。

【练习2】对于广义表 $((a, b), (()), (a, (b)))$ 来说, 下列 () 正确。

A. 长度为4

B. 深度为4

C. 有3个元素

D. 有两个元素

答案: C

**解释: 长度为3, 深度为3, 表头是 (a, b)
表尾是 $((()), (a, (b)))$**

广义表的构成&长度

$A=()$ → 空表，其长度为0，深度为1

$B=(e)$ → 只含有单个原子 e 的表，其长度为1

$C=(x, y, z)$ → 表中每项都是原子，其长度为3

$D=(a, (b, c, d))$ → 有两个元素，一个是原子，
 (b, c, d) 是子表，其长度为2

$E=(A, B, C, D)=((), (e), (x, y, z), (a, (b, c, d)))$

$F=((a, (a, b)), ((a, b), c))$

有4个元素，每个元素又都是一个子表， E 的长度为4

$G=(a, G)$

只含有一个元素，该元素是一个子表， F 的长度为1

a 是原子， G 是一个子表，此广义表长度为2

$G=(a, (a, (a, \dots)))$

【练习3】空的广义表是指广义表（ ）。

- A. 深度为0 B. 尚未赋值
C. 不含任何原子 D. 不含任何元素

答案：D

解释：深度为1，一重括号，不含原子项且不含子表

广义表的表头表尾

$A = () \rightarrow$ 空表无表头表尾

$B = (e) \rightarrow$ 表头= e , 表尾= $()$

$C = (x, y, z) \rightarrow$ 表头= x , 表尾= (y, z)

$D = (a, (b, c, d)) \rightarrow$ 表头= a , 表尾= $((b, c, d))$

$E = (A, B, C, D) = ((), (e), (x, y, z), (a, (b, c, d)))$

$F = ((a, (a, b), ((a, b), c)))$

$G = (a, G)$

表头= $()$, 表尾= $((e), (x, y, z), (a, (b, c, d)))$

表头= $(a, (a, b), ((a, b), c))$,
表尾= $()$

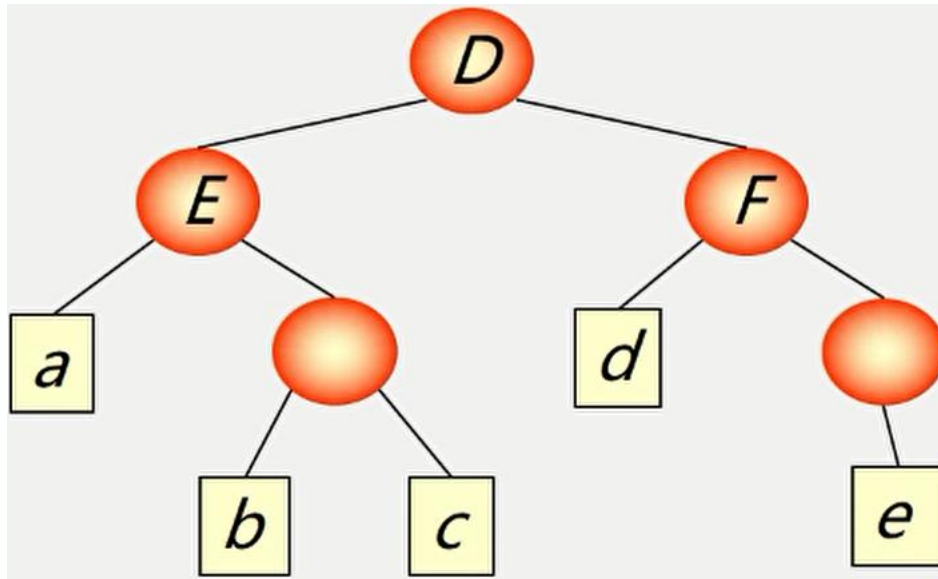
表头= a , 表尾= (G)

4.5.1 广义表(P122)

广义表是多层次结构，广义表的元素可以是单元素，也可以是子表，而子表的元素还可以是子表，...

可以用树形图来表示。

例： $D=(E, F)$ 其中： $E=(a, (b, c))$ $F=(d, (e))$



▶▶▶ 广义表与线性表的区别？



广义表的结构很灵活，它可以兼容线性表、数组、树和有向图等各种数据结构



即线性表是广义表的特例

**【练习4】 广义表 $A=(a, b, (c, d), (e, (f, g)))$,
则 $\text{GetHead}(\text{GetTail}(\text{GetHead}(\text{GetTail}(\text{GetTail}(A))))$
的值为 () 。**

A. (g)

B. (d)

C. c

D. d

$\text{GetTail}(A)=(b, (c, d), (e, (f, g)))$

$\text{GetTail}(\text{GetTail}(A))=((c, d), (e, (f, g)))$

$\text{GetHead}(\text{GetTail}(\text{GetTail}(A)))=(c, d)$

$\text{GetTail}(\text{GetHead}(\text{GetTail}(\text{GetTail}(A))))=(d)$

$\text{GetHead}(\text{GetTail}(\text{GetHead}(\text{GetTail}(\text{GetTail}(A))))= d$

【练习5】 下列4个广义表中，长度为1，深度为4的广义表是（ ）。

A. $((), ((a)))$

B. $(((((a), b))), c)$

C. $(((((a, b), (c))))$

D. $((((a, (b), c))))$

答案： D

解释： A长度为2， B长度为2， C长度为1、深度为3

【练习6】 对于广义表 $LS=(((a), (b)), ((c)))$, 执行以下操作的结果是 ()。

1、 $\text{GetTail}(\text{GetHead}(\text{GetTail}(LS)))=$ _____

2、 $\text{GetHead}(\text{GetTail}(\text{GetHead}(LS)))=$ _____

A. () B. (()) C. (a) D. (b) E. (c)

答案: A、D

解释: 1、 $\text{GetTail}(LS)=(((c)))$ $\text{GetHead}(\text{GetTail}(LS))=((c))$

2、 $\text{GetHead}(LS)=((a),(b))$ $\text{GetTail}(\text{GetHead}(LS))=((b))$

练习一下

1. 广义表 $(a, (a, b), d, e, ((i, j), k))$ 的长度是 5，深度是 3。

2. 广义表 $L = ((), ())$ ，则 $\text{head}(L)$ 是 $(()$ ， $\text{tail}(L)$ 是 $(())$ ， L 的长度是 2。

3. $L = ((\text{cat}, ()), (\text{fish}, (\text{pig}, \text{dog})), \text{bird})$ 。

任务：解救 pig ，将其取出

$\text{head}(\text{head}(\text{tail}(\text{head}(\text{tail}(L)))))$

练习一下

4. 已知广义表: $A=(a, b)$, $B=(A, A)$, $C=(a, (b, A), B)$, 求下列运算的结果: $\text{GetTail}(\text{GetHead}(\text{GetTail}(C))) = (\text{B})$ 。

A. (a) B. (A) C. a D. (b)

5. 广义表 $L=(a, (b, c))$, 进行 $\text{GetTail}(L)$ 操作后的结果为 (D)。

A. c B. b, c C. (b, c) D. ((b, c))

6. 设广义表 $L=((a, b, c))$, 则 L 的长度和深度分别为 (C)。

A. 1和1 B. 1和3 C. 1和2 D. 2和3

1. 了解串的存储方法，重点掌握串模式匹配算法**KMP**算法。
2. 掌握**数组地址**计算方法。
3. 掌握广义表的定义、性质及其**GetHead**和**GetTail**的操作。